

Programação de Sistemas Linux

Aula 07 – Shell Scripting

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br



Onde estamos

Aula 6 (feito)

- BIOS/UEFI, GRUB, kernel, systemd
- Criação de serviços (.service)
- Logs de boot com journalctl

Aula 7 (hoje)

- Automatizar tarefas com **scripts Bash**
- Variáveis, expansões, redirecionamento e *pipes*
- Controle de fluxo e funções
- Argumentos com getopts e sinais com trap
- Filtrar texto com grep/sed/awk



Notes

Notes

Objetivos da aula

Ao final desta aula, você será capaz de:

- ✓ Escrever e executar um **script Bash** do zero
- ✓ Usar **variáveis**, **expansões** e **redirecionamento/pipes**
- ✓ Controlar o fluxo com `if`, `case`, `for` e `while`, e organizar código em **funções**
- ✓ Tratar **argumentos** de linha de comando com `getopts`
- ✓ Reagir a **sinais** e garantir limpeza com `trap`
- ✓ Filtrar e transformar texto com `grep`, `sed` e `awk`



Notes

Fundamentos de um script

Notes

O primeiro script

```
1 #!/bin/bash
2 echo "Ola, $(whoami)! Hoje e $(date +%Y-%m-%d)."
3
```

```
1 $ chmod +x saudacao.sh      # precisa de permissao de execucao
2 $ ./saudacao.sh             # roda usando o shebang da 1a linha
3 $ bash saudacao.sh          # roda explicitamente com bash (nao precisa +
    x)
4
```

- A **shebang** (`#!/bin/bash`) na **primeira linha** diz ao kernel **qual interpretador** usar para rodar o arquivo
- Sem `chmod +x`, `./script.sh` dá “Permission denied”



Notes

Variáveis: sem espaço, com \$

```
1 nome="turma"                # atribuicao: SEM espaco ao redor do =
2 echo "Ola, $nome"           # uso: com $
3 echo "Ola, ${nome}!"        # chaves: evita ambiguidade (ex.: ${nome}s)
4
5 export CAMINHO=/opt/ps1     # variavel de AMBIENTE (visivel p/ processos
    filhos)
6
```

- `nome = "turma"` (com espaços) é um **erro** – bash tenta rodar `nome` como comando
- Aspas **duplas** (`"$var"`) expandem variáveis; aspas **simples** (`'$var'`) não expandem nada



Notes

Expansões: o \$ faz muita coisa

```
1 echo $(date +%H:%M)      # substituição de comando: roda e captura a
   saída
2 echo $((5 + 3 * 2))      # expansão aritmética: 11
3 echo ${#nome}            # tamanho da string em $nome
4 echo *.txt               # globbing: expande para os nomes de arquivo
5 echo ${nome:-"padrao"}   # valor padrao, se $nome estiver vazia/nao-
   definida
6
```

- \$(comando) substitui o estilo antigo ‘comando’ — prefira sempre \$()
- \$((...)) faz **aritmética inteira** — \$(()) não entende ponto flutuante



Notes

Redirecionamento e pipes

Notes

Três fluxos, um símbolo cada

Fluxo	Descritor	Uso comum
stdin (entrada)	0	comando < arquivo
stdout (saída normal)	1	comando > arquivo
stderr (saída de erro)	2	comando 2> erros.log

Todo processo nasce com esses três fluxos abertos — redirecionar é só trocar para onde eles apontam.



Notes

Pipes: a saída de um vira a entrada do outro

```
1 $ ps aux | grep firefox      # filtra a lista de processos
2 $ cat acesso.log | wc -l     # conta linhas
3 $ dmesg | grep -i erro | tail -5 # encadeia varios filtros
4
```

- | conecta o stdout de um comando ao stdin do próximo
- É a mesma ideia dos pipes anônimos que vocês vão ver em C mais adiante na disciplina — aqui é o shell fazendo isso para você
- Um pipeline de 3–4 comandos pequenos costuma ser **mais claro** que um script gigante fazendo tudo



Notes

Estruturas de controle e funções

Notes

if/elif/else

```
1 if [ -f "$1" ]; then
2     echo "$1 e um arquivo regular"
3 elif [ -d "$1" ]; then
4     echo "$1 e um diretorio"
5 else
6     echo "$1 nao existe"
7 fi
8
```

- [...] é o comando test — -f, -d, -z, -eq... são seus testes
- [[...]] (bashismo) aceita &&/|| dentro dos colchetes e não faz *word splitting* — prefira quando não precisar de POSIX puro
- Espaço **obrigatório** depois de [e antes de]

Notes



case: várias opções, sem elif em cascata

```
1 case "$1" in
2     start)
3         echo "Iniciando..." ;;
4     stop)
5         echo "Parando..." ;;
6     restart|reload)
7         echo "Reiniciando..." ;;
8     *)
9         echo "Uso: $0 {start|stop|restart}" ;;
10 esac
11
```

- Padrão *) funciona como o “senão” — pega qualquer coisa que não bateu antes
- opcao1|opcao2) testa **várias** palavras no mesmo bloco



Notes

for e while

```
1 for arquivo in *.txt; do
2     echo "Processando: $arquivo"
3 done
4
5 for i in $(seq 1 5); do echo "$i"; done
6
7 contador=0
8 while [ "$contador" -lt 3 ]; do
9     echo "Tentativa $contador"
10    contador=$((contador + 1))
11 done
12
```

- for var in lista funciona sobre **qualquer lista**: arquivos (globbing), saída de comando, números
- while repete **enquanto** a condição for verdadeira — cuidado com laços infinitos



Notes

```
1 log_erro() {  
2     local mensagem="$1"  
3     echo "[ERRO] $mensagem" >&2  
4 }  
5  
6 processa() {  
7     echo "Script: $0"  
8     echo "Total de argumentos: $#"  
9     echo "Todos os argumentos: $@"  
10    echo "Primeiro: $1 Segundo: $2"  
11 }  
12  
13 log_erro "arquivo nao encontrado"  
14 processa a b c  
15
```

- local evita que a variável **vaze** para fora da função
- \$0, \$1, \$2..., \$#, \$@ — os mesmos que vocês vão usar em qualquer script daqui pra frente



Notes

Argumentos de linha de comando e sinais

Notes

O problema

`meuscript.sh -v -o saida.txt` e `meuscript.sh -o saida.txt -v` deveriam fazer a **mesma coisa** — mas se você só olhar \$1, \$2 na ordem, a posição importa e o código vira um emaranhado de `if`.

`getopts` **resolve isso**:

- Processa opções de **uma letra** (`-v`, `-o` valor) em **qualquer ordem**
- Já vem com tratamento de opção inválida e de argumento faltando
- É um *builtin* do shell — funciona em qualquer sistema com `sh`/`bash`, sem instalar nada



Notes

getopts na prática

```
1 VERBOSE=0
2 OUTFILE="saida.txt"
3
4 while getopts "vo:h" opt; do
5     case "$opt" in
6         v) VERBOSE=1 ;;
7         o) OUTFILE="$OPTARG" ;;
8         h) echo "Uso: $0 [-v] [-o arquivo]"; exit 0 ;;
9         \?) echo "Opcao invalida: -$OPTARG" >&2; exit 1 ;;
10    esac
11 done
12 shift $((OPTIND - 1))
13
```

- `"vo:h"`: `v` e `h` **não** levam valor; `o`: (com `:`) **exige** um valor, disponível em `$OPTARG`
- `shift $((OPTIND - 1))` descarta as opções já processadas, sobrando só os argumentos “normais”



Notes

Definição

Um **signal** é uma notificação assíncrona enviada a um processo — Ctrl+C manda SIGINT, kill manda SIGTERM por padrão. Vamos ver sinais em C mais adiante na disciplina; hoje, o shell já deixa **reagir** a eles com trap.

- Sem trap, Ctrl+C mata o script **na hora** — arquivos temporários, locks, conexões abertas ficam **largados**
- trap deixa você rodar uma função de **limpeza** antes de sair



trap na prática

```
1 ARQUIVO_TMP=$(mktemp)
2
3 limpeza() {
4     echo "Sinal recebido - limpando $ARQUIVO_TMP antes de sair"
5     rm -f "$ARQUIVO_TMP"
6     exit 1
7 }
8
9 trap limpeza SIGINT SIGTERM
10
11 echo "dados importantes" > "$ARQUIVO_TMP"
12 sleep 30 # Ctrl+C aqui deve chamar "limpeza" em vez de matar direto
13
```

- trap funcao SINAL1 SINAL2 ... registra o mesmo tratamento para vários sinais
- Dentro da função, exit **explícito** é necessário — sem ele, o script continuaria de onde parou



trap ... EXIT: limpeza garantida

```
1 ARQUIVO_TMP=$(mktemp)
2 trap 'rm -f "$ARQUIVO_TMP"' EXIT
3
4 echo "trabalhando com $ARQUIVO_TMP..."
5 # nao importa COMO o script termina (fim normal, erro, Ctrl+C) --
6 # o rm de cima roda de qualquer jeito
7
```

- EXIT não é um sinal do kernel — é um **pseudo-sinal** do bash que dispara sempre que o script está prestes a terminar, **de qualquer forma**
- Padrão muito usado para garantir que arquivos temporários **sempre** sejam apagados



Notes

Filtrando texto: grep, sed, awk

Notes

grep: encontrar linhas

```
1 $ grep "ERROR" servidor.log      # linhas que contem "ERROR"
2 $ grep -c "ERROR" servidor.log   # so a CONTAGEM de linhas
3 $ grep -i "erro" servidor.log    # ignora maiusculas/minusculas
4 $ grep -E "[0-9]{4}-[0-9]{2}-[0-9]{2}" servidor.log # regex estendida
5
```

- **grep filtra linhas** — não modifica nada, só decide o que passa
- **-E** liga expressões regulares **estendidas** (+, ?, {n,m}, | sem escapar)



Notes

sed: editar em fluxo

```
1 $ sed 's/,;/g' dados.csv        # troca TODAS as virgulas por ponto-e
   -virgula
2 $ sed -n '2,4p' arquivo.txt     # imprime so as linhas 2 a 4
3 $ sed '1!s/,;/g' dados.csv      # troca, EXCETO na primeira linha (
   cabecalho)
4
```

- **s/padrão/substituição/** é o comando mais usado do sed — *stream editor*
- **g** no final = substitui **todas** as ocorrências na linha, não só a primeira
- Por padrão, sed **não** modifica o arquivo original — some **-i** para editar *in place*



Notes

```
1 $ awk -F',' '{print $1}' dados.csv          # so a 1a coluna (separador:
    virgula)
2 $ awk -F',' 'NR==1{next} {soma+=$2} END{print soma}' dados.csv
3 $ awk -F',' '$2 > 8 {print $1, "aprovado"}' dados.csv
4
```

- \$1, \$2... são as **colunas** da linha atual; NR é o número da linha
- Blocos {...} rodam **por linha**; END{...} roda **uma vez**, no final
- Para tarefas maiores que uma linha de awk, considere escrever um script .awk separado



Regex básica vs. estendida

Símbolo	Significado	Precisa de -E?
.	qualquer caractere	não
*	zero ou mais do anterior	não
+	um ou mais do anterior	sim
?	zero ou um do anterior	sim
{n,m}	entre n e m repetições	sim
	ou (alternância)	sim
^, \$	início/fim da linha	não

Sem -E, +/?/{}/| viram caracteres **literais**, não operadores.



Here-documents

Notes

Here-documents: texto de várias linhas

```
1 cat <<RELATORIO
2 Relatorio gerado para: $NOME
3 Data: $(date +%Y-%m-%d)
4 RELATORIO
5
6 cat <<'LITERAL'
7 Este texto NAO expande variaveis: $NOME fica assim mesmo.
8 LITERAL
9
```

- **DELIMITADOR** **expande** variáveis e `$(...)`, como texto normal com aspas duplas
- **'DELIMITADOR'** (com aspas) **não expande nada** — útil para gerar templates, scripts, JSON literal
- O delimitador (RELATORIO, LITERAL...) pode ser qualquer palavra — só precisa fechar igual abriu

Notes



Encerramento

Notes

Síntese da aula

Recurso	Para que serve
#!/bin/bash + chmod +x	Tornar o script executável
set -euo pipefail	Parar no primeiro erro, em vez de seguir errado
"\$VAR", \$(cmd)	Expandir variáveis e capturar saída de comando
>, >&, 2>,	Redirecionar fluxos e encadear comandos
if, case, for, while	Controlar o fluxo de execução
Funções + \$1, \$@	Reaproveitar código dentro do script
getopts	Tratar opções de linha de comando
trap	Reagir a sinais e garantir limpeza
grep / sed / awk	Buscar / editar em fluxo / processar colunas

A escolha entre os três filtros

`grep` seleciona linhas, `sed` edita texto em fluxo e `awk` trabalha com campos e agregações. Na dúvida: se precisa somar ou agrupar por coluna, é `awk`.

Notes



Laboratório 06 — Shell Scripting (2h)

Parte A, em aula:

- 1 Script de **backup** com compressão e *timestamp*, tratando erros de uso
- 2 Script de **deploy** com opções de linha de comando e validação
- 3 *Pipeline* com `grep`, `sed` e `awk` sobre um CSV

Parte B, extraclasse: analisador de log, monitor de recursos com `trap` e um desafio de `awk`.



That's all folks!



**INSTITUTO
FEDERAL**
Santa Catarina
Câmpus
Florianópolis



Notes

Notes
